



Année académique 2024-2025

Elément de module : Complexité algorithmique et structures de données
Examen terminal durée : 90 minutes documents non autorisés

On peut utiliser et appeler les fonctions ou algorithmes classiques de manipulation de structures de données sans les réimplémenter

Exercice 1 : Piles et files (10 points)

Nous disposons d'une file f qui contient n piles $P_1, P_2, \dots, P_n$.

- L'ajout d'un élément à cette structure consiste à l'empiler sur la dernière pile de la file.
 - La suppression consiste à dépiler l'élément de la première pile de la file.
1. Écrire l'algorithme ou la fonction **EnfilerDepiler** qui prend un entier en entrée et l'insère dans cette structure.
 2. Écrire l'algorithme ou la fonction **DefilerDepiler** qui dépile un élément de la première pile de la file.
 3. On souhaite construire, à partir d'une file de n piles, une nouvelle file contenant 2 piles en fonction d'un paramètre e .
 - La première pile contient tous les éléments des n piles dont la valeur est inférieure ou égale à e .
 - La deuxième pile contient tous les éléments des n piles dont la valeur est strictement supérieure à e .

Écrire l'algorithme qui prend en entrée une file de n piles et retourne la file à 2 piles. Puis, donner la complexité de votre algorithme en fonction du nombre total d'éléments dans les n piles.

Exercice 2 : Arbre binaires (10 points)

On considère la structure de données suivante :

```
struct etudiant{  
    int matricule ;  
    String nom ;  
    String prenom ;  
    String specialite ;  
};
```

On souhaite manipuler une liste d'étudiants à l'aide d'un arbre binaire de recherche dont la clé est le matricule (qui est unique).

1. Décrire la structure de l'arbre binaire de recherche en tenant compte du fait que les éléments stockés sont des étudiants.
2. Écrire un algorithme qui permet d'insérer un nouvel étudiant dans l'arbre binaire de recherche.
3. On appelle **SelectById** la fonction qui permet d'afficher toutes les informations d'un étudiant en fournissant son matricule. Écrire le code de cette fonction.
4. On appelle **SelectAllOrderByid** la fonction qui permet d'afficher la liste de tous les étudiants triés par ordre croissant de leur matricule. Écrire cette fonction.

, ent val